ANTIGRAVITY EXTENSIBILITY ARCHITECTURE

안티그래비티 환경 종합 아키텍처 보고서

[제2권] 커스터마이징 5대 기둥, 점진적 공개, MCP 표준 프로토콜 및 아티팩트 엔진

SPECIFICATION	EXTENSION PILLARS	TARGET PROJECT	PUBLICATION DATE	ARCHITECT
AGY Customization Spec 2.0	Rules / Skills / Plugins / Hooks / MCP	FC (Unreal Engine 5.8 C++)	2026-10-22	Lead Systems & AI Architect

1. 개요 및 커스터마이징 철학 (Customization Philosophy)

안티그래비티의 핵심 설계 목표는 범용 대규모 언어 모델(LLM)을 **프로젝트 고유의 아키텍처와 컨벤션에 완전히 동기화된 도메인 특화 전문가(Domain-Specific Expert)**로 전환하는 것이다. 이를 위해 시스템은 단순 프롬프트 주입 방식을 탈피하여 **규칙(Rules), 기술(Skills), 플러그인(Plugins), 훅(Hooks), 모델 컨텍스트 프로토콜(MCP)**의 **5대 확장 기둥(The 5 Pillars)**을 제공한다.

1. Rules
불변 아키텍처 원칙, 코딩 표준, API 제약 조건의 계층적 강제 (AGENTS.md, GEMINI.md).

2. Skills
다단계 절차, 런북, 스크립트 실행 워크플로를 온디맨드로 캡슐화한 절차적 지식 단위.

3. Plugins
연관된 스킬, 규칙, MCP 구성을 단일 네임스페이스 단위로 번들링하여 배포/관리.

4. Hooks
에이전트 수명주기 이벤트(도구 호출 전/후 등)를 가로채어 정책 검증 및 보안 감사 자동화.

5. MCP Servers
외부 도구, 데이터베이스, 언리얼 엔진 런타임을 표준 JSON-RPC로 결합하는 오픈 프로토콜.

2. 커스터마이징 생태계 통합 토폴로지 (Ecosystem Architecture)

[Antigravity 5-Pillar Customization & Context Ingestion Pipeline]

[Rules] Hierarchical & Path-Walk | **[Skills]** Progressive Disclosure | **[Plugins]** Bundled Unit & Manifest | **[Hooks]** Lifecycle Interception | **[MCP Servers]** Live Engine RPC Bridge

↓ **Discovery Engine & Deduplication Resolver** (Precedence Order Evaluation) ↓

Context Window Injection
선별적 프롬프트 주입 (토큰 최적화)

Artifact Engine
구조화된 다이어그램 / 마크다운 문서 생성

Session Transcripts
JSONL 기반 상태 추적 및 /compact 압축

3. 5대 기둥의 핵심 특성 비교 매트릭스

기둥 (Pillar)	구성 파일 / 폴더	적용 스코프	주요 역할 및 강제 사항	로드 타이밍
Rules	AGENTS.md, GEMINI.md	프로젝트 전역 / 디렉터리	아키텍처 불변 원칙, 코딩 표준, 금지 패턴 강제	상시 또는 모델 판단
Skills	skills/<name>/SKILL.md	온디맨드 (Progressive)	복잡한 다단계 워크플로, 빌드/테스트 스크립트 실행 절차	동적 점진적 로드
Plugins	plugins/<name>/plugin.json	네임스페이스 번들	팀 단위 스킬셋, 룰, MCP 도구 구성을 단일 패키지로 제공	설치/활성화 시
Hooks	hooks.json	수명주기 이벤트	도구 실행 전/후 린터 검사, 보안 차단, 감사 로그 기록	이벤트 발생 시점
MCP Servers	mcp_config.json	도구/리소스 통합	언리얼 라이브 에디터, DB, 원격 API와의 양방향 JSON-RPC 통신	도구 디스패치 시

1. 탐색 우선순위 (Loading Priority & Precedence Order)

안티그래비티는 여러 위치에서 동일한 이름의 스킬, 규칙, 도구가 발견될 때 명확한 우선순위 규칙에 따라 충돌을 해결한다. **더 구체적인 프로젝트 레벨 구성이 전역 머신 구성을 항상 오버라이드**한다.

5단계 우선순위 계층도 (Highest → Lowest)

1. **1. Workspace Project (최우선)**
   작업 디렉터리(CWD)부터 Git 루트까지 상향 탐색하여 발견된 .agents/, .antigravity/, AGENTS.md.
2. **2. Declared Configurations**
   워크스페이스 내부 skills.json 또는 plugins.json에 명시적으로 등록된 외부 경로 커스터마이징.
3. **3. Global Discovery**
   개발자 로컬 머신의 사용자 홈 디렉터리 ~/.gemini/config/ 하위의 프로젝트 비종속 설정.
4. **4. Built-in Customizations**
   안티그래비티 애플리케이션 바이너리에 기본 탑재된 빌트인 스킬 및 시스템 프롬프트.
5. **5. Global Declared Configurations (최하위)**
   머신 전역 JSON 설정에 등록된 외부 참조 커스터마이징.

탐색 및 해결 메커니즘 다이어그램

```
# Customization Resolution Hierarchy [Local CWD: FC/Source/FC/Combat] |-- Walk-up → FC/ (Found .antigravity/rules, AGENTS.md) [Priority 1] |-- Walk-up → Git Root (.git) → Stop traversal v [Workspace Manifests: skills.json] [Priority 2] v [Global Dir: ~/.gemini/config/] [Priority 3] v [Built-in Runtime Defaults] [Priority 4] Outcome: Project rules (AGENTS.md) always override global defaults!
```

2. 점진적 공개 메커니즘 (Progressive Disclosure)

[컨텍스트 최적화 핵심] Progressive Disclosure 원리 Context Preservation

수십 개의 스킬과 수백 줄의 룰셋을 세션 시작 시 한꺼번에 컨텍스트 윈도우에 주입하면, 모델의 유효 토큰이 고갈되고 환각(Hallucination) 위험이 극대화된다. 안티그래비티는 **메타데이터 요약본만 초기 주입하고, 실제 필요 시에만 본문을 주입**하는 점진적 공개 아키텍처를 채택한다.

1단계: 경량 메타데이터 주입 (Lightweight Catalog) Initial Turn

세션 초기화 시 각 스킬의 name과 description(1~2줄 요약)만을 시스템 프롬프트의 카탈로그 영역에 등록한다. 토큰 소모량은 스킬당 수십 토큰 미만으로 극소화된다.

2단계: 온디맨드 심층 로드 (On-Demand Deep Load) Dynamic Activation

사용자의 요청이 해당 스킬의 설명과 일치하거나 에이전트가 절차적 지식을 필요로 할 때, 에이전트 런타임이 SKILL.md 및 참조 문서(references/) 전문을 해당 턴의 컨텍스트로 지연 로드(Lazy Load)한다.

3. 경로 기반 중복 제거 (Path-Based Deduplication)

여러 규칙 디렉터리나 트리거 조건이 동일한 파일 경로를 가리키더라도, 런타임의 **Deduplication Engine**이 절대 정규화 경로(Canonical Resolved Path)를 기준으로 중복을 필터링한다. 단일 대화 턴 내에서 동일한 룰셋이나 스킬이 중복 인젝션되는 현상은 시스템 레벨에서 원천적으로 방지된다.

1. 규칙(Rules) 아키텍처 & 디렉터리 계층화

규칙(Rules)은 에이전트가 코드를 작성하거나 수정할 때 반드시 준수해야 하는 **불변의 제약 조건(Constraints)과 컨벤션**을 정의한다.

1. 전역/프로젝트 최상위 규칙 (AGENTS.md) Supreme Directives

프로젝트 루트의 AGENTS.md는 모든 하위 작업에 상시 적용되는 최고 권위 지침서이다. 네트워크 4계층 권한 분리, Server Authority 준수, Cooked Asset SSOT, 최소 수정 원칙을 정의한다.

2. 모듈러 도메인 룰셋 (.antigravity/rules/*.md) Domain-Specific

C++20 코딩 표준, 멀티플레이어 컨벤션, 루프 엔지니어링, GAS 속성, 인벤토리 네트워킹, MVVM 등 특정 도메인 영역 작업 시에만 선별적으로 활성화되는 전문 규칙 모듈이다.

2. 규칙 파일 포맷 및 트리거 메타데이터

```
--- trigger: always_on # or trigger: model_decision (온디맨드 점진적 로드) description: "언리얼 엔진 5.8 C++20 코딩 표준 및 메모리 관리 지침" paths: - "Source/**/*.h" - "Source/**/*.cpp" --- # Rule Content (Markdown) - TObjectPtr<T> 사용 의무화 (원시 포인터 UPROPERTY 금지) - IWYU (Include-What-You-Use) 준수 및 전방 선언(Forward Declaration) 적극 활용
```

3. 기술(Skills) 구조 및 다단계 절차 캡슐화

스킬(Skills)은 복잡한 다단계 워크플로, 디버깅 절차, 도구 실행 체인을 캡슐화하여 에이전트에게 **'방법(How-To)'**을 교육하는 실행형 지식 모듈이다.

스킬 디렉터리 표준 구조

```
skills/<skill-name>/ |-- SKILL.md # 메인 지침 및 YAML 프론트매터 (필수) |-- scripts/ # 자동화 보조 파이썬/배치 스크립트 |-- references/ # 오프라인 참조 하위 문서 (Markdown) |-- examples/ # 실제 코드 예제 및 입출력 템플릿 +-- resources/ # 에셋 템플릿, 사전 컴파일 바이너리
```

SKILL.md 프론트매터 표준

```
--- name: ue5-gas-generator description: >- 언리얼 엔진 5.8 GameplayAbility, AttributeSet, GameplayEffect 보일러플레이트를 생성하고 복제 검증을 수행. --- # Multi-Step Procedure: 1. Define FGameplayAttributeData in Header 2. Implement OnRep functions with RepNotify 3. Run python Scripts/pipeline.py static for verification
```

[규칙과 스킬의 상호보완적 결합] The Rules + Skills Synergy Architecture Cohesion

규칙(Rules)이 "서버 전용 함수는 반드시 HasAuthority()로 감싸야 한다"는 **불변의 원칙(What/Constraint)**을 정의한다면, **스킬(Skills)**은 "전투 능력 구현 시 AttributeSet 생성부터 OnRep 바인딩 및 파이프라인 검증까지 단계별로 어떻게 수행하는가"의 **절차적 실행(How-To/Workflow)**을 이끈다. 이 둘의 결합을 통해 에이전트는 결함 없는 고품질 코드를 자율 작성한다.

1. 플러그인(Plugins) 번들링 아키텍처

플러그인(Plugins)은 연관된 스킬, 규칙, MCP 서버 설정 및 라이프사이클 훅을 **단일 배포 단위로 캡슐화한 네임스페이스 모듈**이다. 팀 단위로 표준화된 도구셋을 형상 관리(VCS)하고 프로젝트 간에 손쉽게 공유할 수 있도록 지원한다.

플러그인 매니페스트 (plugin.json) 구조

```
{ "name": "fc-unreal-toolkit", "version": "2.1.0", "description":
"FC UE5.8 게임플레이 및 파이프라인 전용 플러그인", "skills":
["skills/ue5-build", "skills/asset-inspect"], "rules": ["rules/01-
ue58-coding-standards.md"], "mcp_servers": "mcp_config.json",
"hooks": "hooks.json" }
```

플러그인 관리 및 슬래시 커맨드

- **/plugin install <path>:** 로컬 또는 원격 리포지토리의 플러그인을 워크스페이스에 즉시 설치.
- **/plugin enable/disable:** 현재 세션에서 특정 플러그인의 스킬/룰셋을 동적으로 활성화/비활성화.
- **네임스페이스 충돌 격리:** 각 플러그인은 고유 네임스페이스를 부여받아 다른 플러그인의 동일 이름 스킬과 상호 충돌하지 않음.

2. 라이프사이클 훅(Hooks) 엔진

훅(Hooks)은 에이전트의 런타임 수명주기 이벤트 시점에 외부 스크립트나 명령어를 가로채어 실행(Interception)하는 거버넌스 자동화 계층이다.

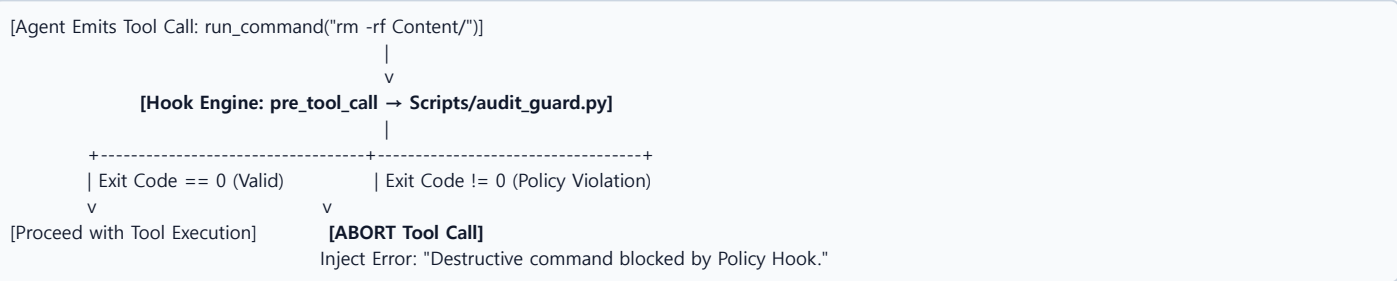
pre_tool_call
도구가 실행되기 직전 호출. 파괴적 명령어 사전 차단, 보안 정책 검증, 린터 사전 점검.

post_tool_call
도구 실행 직후 호출. 표준 출력 로그 수집, 파일 변경 이력 Git 자동 스테이징.

pre_message
사용자 입력 처리 직전 호출. 민감 정보(API 키 등) 마스킹 및 추가 컨텍스트 주입.

session_start
세션 초기화 시점 호출. 환경 변수 검증, 빌드 도구 바이너리 설치 상태 점검.

3. 훅 인터셉션 흐름도 및 hooks.json 설정 스펙



[보안 거버넌스] 무결성 보장을 위한 훅 Fail-Closed 원칙 Fail-Closed Security

사전 검증 훅(pre_tool_call) 스크립트 실행 중 파이썬 예외가 발생하거나 타임아웃이 초과되면, 시스템은 안전을 위해 해당 도구의 실행을 **기본 차단(Fail-Closed)**한다. 이를 통해 예기치 않은 스크립트 버그로 인해 비검증 위험 명령어가 시스템에 전파되는 사고를 원천 방지한다.

1. Model Context Protocol (MCP) 개요 및 표준화

Model Context Protocol (MCP)은 AI 에이전트와 외부 시스템(개발 도구, 데이터베이스, 게임 엔진) 간의 상호작용을 표준화하기 위해 설계된 **개방형 프로토콜(Open Standard)**이다. 에이전트가 각 도구의 내부 구현을 몰라도 일관된 JSON-RPC 규격을 통해 기능과 데이터를 탐색하고 안전하게 호출할 수 있다.

1. Tools (실행 함수)
에이전트가 매개변수를 전달하여 환경을 변이시키거나 작업을 수행하도록 노출된 실행형 RPC 함수 (예: 언리얼 액터 스폰).

2. Resources (데이터 피드)
에이전트가 읽을 수 있는 파일, 로그 스트림, DB 테이블 스키마 등 실시간 정적/동적 데이터 소스 (URI 기반 라우팅).

3. Prompts (프롬프트 템플릿)
서버가 클라이언트에게 제공하는 구조화된 사전 정의 프롬프트 템플릿 및 인터랙티브 워크플로 매크로.

2. MCP 전송 계층 (Transport Layers) 및 설정 표준

전송 계층 비교 (stdio vs sse)

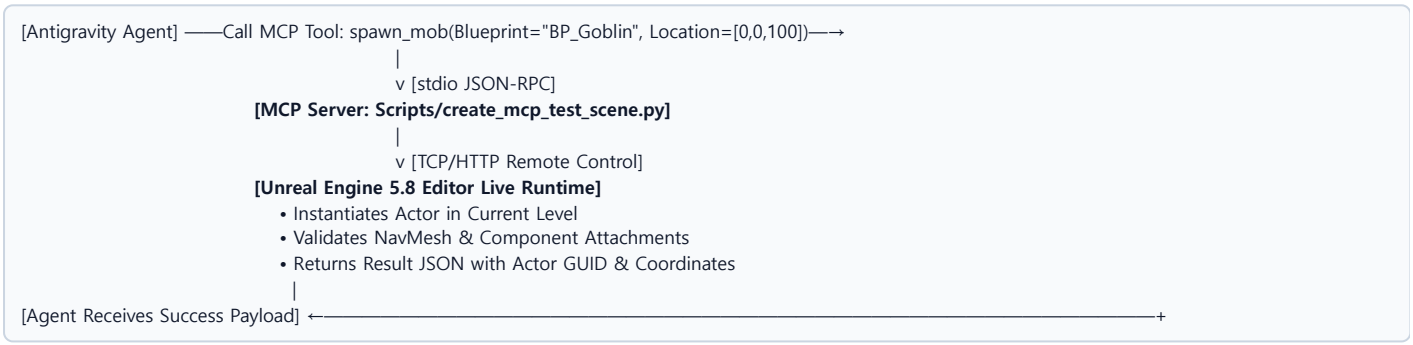
- **stdio (표준 입출력):** 로컬 머신에서 서브프로세스를 스폰하고 파이프(stdin/stdout)를 통해 초고속 JSON 통신 수행. 로컬 도구에 최적.
- **sse (Server-Sent Events):** HTTP 기반으로 원격 도커 컨테이너, 클라우드 인프라 또는 분산 서비스와 이벤트 스트림 통신.

mcp_config.json 설정 스키마

```
{ "mcpServers": { "unreal-live-bridge": { "command": "python",
"args": ["Scripts/mcp_unreal_bridge.py"], "env": { "UE_PORT": "30010"
} } } }
```

3. FC 언리얼 엔진 5.8 라이브 에디터 연동 아키텍처

안티그래비티는 MCP 브릿지를 통해 언리얼 에디터의 Live Remote Execution API와 연결되어 **실시간 레벨 뷰포트 조작, 액터 배치, 에셋 검사**를 자율 수행한다.



[MCP의 가치] 에디터 재시작 없는 실시간 라이브 튜닝 Zero-Restart Velocity

전통적인 엔진 개발에서는 사소한 파라미터 변경이나 씬 구성을 위해 에디터를 재시작해야 했다. 안티그래비티 MCP 브릿지를 활용하면 에이전트가 실행 중인 에디터 내부의 블루프린트 변수, 카메라 앵글, 스포너 배치를 원격으로 직접 쿼리하고 변경하므로 개발 속도가 비약적으로 향상된다.

1. 아티팩트(Artifacts) 엔진 아키텍처

아티팩트(Artifacts)는 단순 대화형 말풍선 텍스트의 한계를 극복하고, **구조화된 보고서, 설계 다이어그램, 태스크 목록, 코드 Diff**를 영속적으로 기록/렌더링하기 위한 특수 마크다운 문서 시스템이다.

1. 저장소 격리 및 메타데이터 바인딩 Dedicated Storage

모든 아티팩트는 <appDataDir>□rain₩<conversation-id> 전용 디렉터리에 독립 .md 파일로 저장된다. 생성 시 ArtifactMetadata(Summary, UserFacing, RequestFeedback)가 필수 바인딩된다.

2. 대화 컨텍스트 비대화 방지 Context Decoupling

아티팩트를 생성한 후 에이전트는 대화창에 아티팩트 전문을 중복 출력하지 않고 링크만을 제시함으로써 컨텍스트 윈도우 토큰을 비약적으로 보존한다.

2. 아티팩트 지원 렌더링 확장 기능

확장 기능	문법 및 사용 형식	시각화 효과 및 활용 사례
GitHub Alerts	> [!NOTE], > [!TIP], > [!IMPORTANT], > [!WARNING]	배경색과 전용 아이콘이 적용된 시각적 콜아웃 박스 (경고, 중요 사항 강조).
Mermaid Diagrams	```mermaid ... ``` (flowchart, sequenceDiagram, stateDiagram, erDiagram)	인라인 렌더링되는 인터랙티브 벡터 흐름도 및 시퀀스 다이어그램.
Four-Backtick Carousels	````carousel ... <!-- slide --> ````	비포/애프터 코드 비교, 단계별 튜토리얼을 슬라이드 형태로 순차 탐색.
KaTeX Math	인라인 ₩(...₩), $...$ 또는 디스플레이 $$...$$	전투 대미지 공식, TTK 기대값 계산식 등 완벽한 수학 기호 렌더링.

3. 인터랙티브 컴포넌트: ask_question & Generative UI

대화형 질문 모달 (ask_question)

요구사항이 모호하거나 아키텍처 선택지가 갈릴 때, 사용자가 즉시 선택할 수 있는 **인터랙티브 모달 UI**를 렌더링하고 사용자 응답 시까지 안전하게 블로킹한다.

```
ask_question(questions=[{ "question": "인벤토리 네트워킹 방식을 선택하십시오.", "options": [ "(Recommended) FFastArraySerializer 기반 대역폭 최적화", "표준 TArray OnRep 단순 복제" ], "is_multi_select": false }])
```

Generative UI & 스크래치 패드

단순 마크다운을 넘어 풍부한 HTML/CSS 인터랙티브 컨트롤, 실시간 파트 차트, 데이터 테이블을 인라인 또는 독립 패널에 직접 렌더링.

- **Scratch Directory:** 디버깅용 임시 스크립트 및 테스트 데이터를 <conversation-id>/scratch/에 영속 저장.
- **Interactive Approval:** RequestFeedback: true 설정을 통해 사용자의 1-클릭 실행 승인(Proceed) 획득.

1. 세션 트랜스크립트(Transcripts) 이중화 구조

안티그래비티는 에이전트와 사용자의 모든 대화 단계, 사고 과정(Thinking), 도구 호출 이력을 **JSON Lines (JSONL)** 포맷으로 정밀하게 영속화한다.

로그 파일	데이터 특성 및 구조	설계 목적 및 최적화 기법
transcript.jsonl (Compact Log)	대규모 내용(content, thinking, tool_calls)이 truncated_fields로 축약된 경량 스트림.	고속 탐색, 서브에이전트 이력 스캔 및 대화 흐름 개요를 최소 I/O로 빠르게 조회.
transcript_full.jsonl (Full Log)	모든 프롬프트, 도구 페이로드, 수십 KB의 표준 출력이 생략 없이 보존된 완전체 로그.	축약된 필드의 원본 데이터 정밀 복원 및 세션 간 완전한 상태 리플레이 지원.

2. 메모리 최적화 및 슬래시 커맨드 생태계

컨텍스트 압축 (/compact) — Context Compaction

대화가 장기화되어 컨텍스트 윈도우가 임계치에 도달할 때, /compact 커맨드를 실행하여 과거의 사소한 도구 출력과 질의응답을 정밀한 상태 요약본으로 압축하고 토큰을 대폭 확보한다.

목표 지향 자율 실행 (/goal) — Long-Running Autonomy

야간 대규모 리팩토링이나 풀 빌드 검증 시 에이전트가 중도에 멈추지 않고 최종 목표가 100% 달성될 때까지 자율적으로 서브태스크를 연속 해결하도록 보장한다.

3. 제2권 종합 결론 및 제3권 연계 안내

본 제2권에서는 안티그래비티의 커스터마이징 5대 기둥(Rules, Skills, Plugins, Hooks, MCP)과 점진적 공개, 아티팩트 엔진 및 트랜스크립트 상태 관리 메커니즘을 상세히 규명하였다. 이러한 확장성 프레임워크는 대규모 언리얼 엔진 프로젝트인 **FC**의 복합적인 멀티플레이어 아키텍처와 결합하여 비로소 최고의 시너지를 발휘한다.

[Next Volume] 제3권: FC 프로젝트 언리얼 5.8 실전 엔지니어링 & 자율 루프 — Volume 3 Preview

- **4계층 권한 분리 모델:** Server Authority, Replicated State, Autonomous Client, Simulated UI 실전 구현.
- **모듈러 도메인 룰셋 01~06:** C++20 코딩 표준, GAS AttributeSet, Fast Array 직렬화, UMG MVVM 연동 규격.
- **자율 파이프라인 러너:** pipeline.py 7대 모드 및 UBT 컴파일 에러 5회 자율 셀프 힐링(Self-Healing) 루프 완벽 분석.